SEP420 – Web Programming and Scripting

Assignment 1 (10%)

For this assignment, you may work individually or with one partner. If you choose to work with a partner, ensure that both names are included on the submission at the time it is submitted. Names added after submission will not be accepted. Only one member of the group needs to submit the assignment.

In this assignment, you will create a JavaScript program that executes using NodeJS.

When creating your solution, you must use the coding practices and conventions demonstrated in class. A solution that does not reflect what was taught in class will not be accepted (0 grade) and/or be subject to an academic integrity review.

Coding Guidelines

- Variables must be declared with let and const keywords. Var keyword is not permitted
- Functions must be declared using arrow function syntax
- Objects must be represented using Javascript object literals

Submission Requirements

For this submission to be graded, you must submit:

1. A zip file containing your NodeJS project and solution code

How to create your zip file:

1. Create a NodeJS project and write your code:
   - ❑ Create a folder on your computer called **PARKING_FIRSTNAME**. Replace **FIRSTNAME**with your first name(s), example: **PARKING_ALISHA**
   - ❑ Initialize your folder as a NodeJS project using: **npm init −y**
   - ❑ Add your Javascript file to the NodeJS project folder. Name your Javascript file: **app.js**

2. When you are ready to submit, create a zip file of your project:
   - ❑ Create a zip file containing the project folder you created in step 1 above
   - ❑ Rename your zip file to **PARKING_FIRSTNAME.zip**. Replace **FIRSTNAME**with your first name, example:

     PARKING_ALISHA.zip

Academic Integrity

You are permitted to:

- Utilize posted class materials
- Search the internet for Javascript syntax. Ensure you use syntax that aligns with what is taught in this section of the course

You are *not* permitted to:

- Exchange resources and solutions
- Post exercises to or use homework help sites (CourseHero, Chegg, etc) / AI powered tools to generate partial or full code solutions

Problem Description

You have been hired by the college to write a program to help manage parking in the college parking lots. The college needs the program to:

- Allow customers to purchase parking at the college's parking lots
- Generate a report summarizing each parking lot and the amount of parking fees collected.

A customer successfully purchase parking if:

- They choose a valid parking lot (example: If the college has "Parking Lot A" and "Parking Lot B", and you attempt to purchase parking at "Parking Lot C", then it won't work)

  AND

- They are NOT already parked in the parking lot (example: If you are already parked in "Parking Lot A", then you cannot park again)

Creating Your Program

In your JavaScript program, write the code to:

1. Create a **list (array)** of parking lots.
   a. Populate your data structure with 2 parking lots using values of your choice.

2. Create functions for:
   a. enabling customers to purchase parking (purchaseParking())
   b. outputting a summary of the parking lot (printSummary())

3. Using the data structures and functions, write the code to:
   a. Attempt to add 3 license plates to parking lot #1. Ensure you use a variety of values for the license plate and number of hours. If your functions are implemented correctly, your program should output success messages.
   b. Attempt to add 2 license plates to parking lot #2. Ensure you use a variety of values for the license plate and number of hours. If your functions are implemented correctly, your program should output success messages.
   c. Attempt to add a duplicate license plate to parking lot #2. If your functions are implemented correctly, then your program should output an error message.
   d. Attempt to add a license plate to a parking lot that does not exist. If your functions are implemented correctly, then your program should output an error message.
   e. Output a report displaying a summary of the parking lots.

A sample output is provided below.

Data Structure - Parking Lots

In this program, you will need to create data structures for:

- A parking lot
- A car that is parked in a parking lot.

These data structures must be implemented using **Javascript object literals** according to the following specifications:

- Parking Lot

  Each parking lot object literal must contain properties for:
  a. name of parking lot → example: "Short Term Lot"
  b. hourlyRate → cost to park at the lot for 1 hour (example: $2.50)
  c. feesCollected → total amount of money collected at the parking lot
  d. parkedCars → a **list of cars** parked in the lot. Initially, there are no cars in the lot.

- Cars

  The parking lot must contain a list of cars parked in the lot. Each car object literal must contain properties for:
  a. licensePlate → the license plate of the car
  b. hoursParked → how many hours of parking did the customer purchase?
  c. amountPaid → the total amount paid for parking (hoursParked x lot's hourly rate)

Function Definitions

The program must create and implement the following functions:

purchaseParking(*customerLicensePlate, requestedParkingLotName, numberOfHours*)

- This function is used to purchase parking. Specifically, the function should:
  1. Check if the customer's license plate can be added to the requested parking lot
  2. If not, then display an error message indicating the reasons why the license plate cannot be added.
  3. If yes, then:
     a. Add the license plate to the parking lot
     b. Update the total amount of fees collected by the specified parking lot

- The customer's license plate, requested parking lot, and number of purchased parking hours must be provided by the function's parameters.

- <u>Can A License Plate Be Added?</u>

  A license plate can be added to the parking lot if:

  1. The request parking lot EXISTS in the list of parking lots AND
  2. The car with the same license plate DOES NOT EXIST in the specified parking lot's list of parked cars

  You should implement these checks *before* adding the license plate. For example:

  - If the requested parking lot cannot be found, then display an error message and exit the function. Example of appropriate error message:

  ERROR: Cannot park CIVIC808. Sports Stadium Parking Lot does not exist.

  - If the car is already parked in the lot, display an error message and exit the function. Example of appropriate error message:

  ERROR: Cannot add CIVIC808. CIVIC808 is already parked in Garage Parking

  - If the code successfully passes both these checks, *then* write the code to add the license plate to the parking lot.

- <u>How to check if a parking lot or car exists?</u>

  To check if a parking lot or car exists, your code must use a use a for-in or C-style loop to iterate through the **appropriate array** to search for a matching item.

  You are **<u>not permitted</u>** to use while loops or higher order Javascript array functions such as: map, reduce, filter, forEach, contains, find, some, every, flatMap, etc

- <u>Adding a License Plate To the Requested Parking Lot</u>

  If a license plate can be added:

  1. Create a new ***car object literal*** add it to the specified parking lot's **parkedCars** list. Ensure you fill in values for each of the object's properties.
  2. Increase the total fees collected by the purchase amount
  3. Display an appropriate success message to the terminal, for example:

  SUCCESS: License Plate AMG905 is parked in Short Term Lot for 3 hours.

printSummary()

- This function is used to output a report displaying a summary of each parking lot.

- For each parking lot,
  display:

1. The name of the lot
2. The total number of parked cars
3. The hourly rate, displayed to 2 decimal points (use .toFixed())
4. The total fees collected, displayed to 2 decimal points (use .toFixed())
5. The list of cars parked in the lot. For each car, show the license plate and number of hours purchased.

- For example, if there are 2 items in the parking lot list, then the report should look like this.

---- REPORT FOR Short Term Parking ---
-- Number of parked cars: 2

Hourly Rate: $5.00

Total Fees Collected: $65.00
Parked Cars List:

License Plate CHEVY61, Hours Purchased: 3

License Plate FASTCAR722, Hours Purchased: 10

---- REPORT FOR Garage Parking -----
Number of parked cars: 3

Hourly Rate: $2.00

Total Fees Collected: $26.00
Parked Cars List:

License Plate FORD101, Hours
Purchased: 5 License Plate CIVIC808,
Hours Purchased: 7 License Plate
TESLA155, Hours Purchased: 1

END OF ASSESSMENT

<u>Sample Output:</u>

Here is a sample of the expected program output.

You should customize the values of the license plates, hours requested, and parking lots.

Your output should produce similar information (pay attention to what information is shown in the success/error messages)

If you cannot see the above an image, here is the text-format of the output:

SUCCESS: CHEVY61 parked in Short Term Parking for 3 hours. You paid: $15.00

SUCCESS: FORD101 parked in Short Term Parking for 5 hours. You paid: $25.00

SUCCESS: CIVIC808 parked in Garage Parking for 7 hours. You paid: $14.00

SUCCESS: TESLA155 parked in Garage Parking for 1 hours. You paid: $2.00

SUCCESS: FASTCAR722 parked in Garage Parking for 10 hours. You paid: $20.00

ERROR: Cannot add CIVIC808. CIVIC808 is already parked in Garage Parking

ERROR: Cannot park CIVIC808. Sports Stadium Parking Lot does not exist.

---- REPORT FOR Short Term Parking -----

Number of parked cars: 2

Hourly Rate: $5.00

Total Fees Collected: $40.00

Parked Cars List:

License Plate CHEVY61, Hours Purchased: 3

License Plate FORD101, Hours Purchased: 5

---- REPORT FOR Garage Parking -----

Number of parked cars: 3

Hourly Rate: $2.00

Total Fees Collected: $36.00

Parked Cars List:

License Plate CIVIC808, Hours Purchased: 7

License Plate TESLA155, Hours Purchased: 1

License Plate FASTCAR722, Hours Purchased: 10

Late Submission Policy

Assessments submitted after the deadline will incur a penalty of 10% per calendar day, including weekends. Submissions more than 5 days late will not be accepted without prior approval.